

用于正则测试的文本

```
(kali㉿ 20232311zfz)-[~/Desktop]
$ nano test_regex.txt

(kali㉿ 20232311zfz)-[~/Desktop]
$ cat test_regex.txt
user1:13812345678:2026-03-16:login_success
user2:15998765432:2026-03-16:login_fail
user3:18611112222:2026-03-17:login_success
user_admin:020-88889999:2026-03-16:admin_login
invalid_user::2026-03-16:error
Product_A:price_999:stock_50
Product_B:price_1999:stock_5
Product_C:price_299:stock_0
Hello World! 12345 test@example.com
这是中文测试 123abc 空格 制表符      换行
line1
line2
line3
```

1: 基础匹配

(1) 匹配以 user 开头，后接 1 个任意字符的行（匹配任意单个字符）

```
(kali㉿ 20232311zfz)-[~/Desktop]
$ grep 'user.' test_regex.txt
user1:13812345678:2026-03-16:login_success
user2:15998765432:2026-03-16:login_fail
user3:18611112222:2026-03-17:login_success
user_admin:020-88889999:2026-03-16:admin_login
invalid_user::2026-03-16:error
```

(2) 匹配包含 11 位手机号（纯数字）的行（匹配数字）

```
(kali㉿ 20232311zfz)-[~/Desktop]
$ grep -P '\d{11}' test_regex.txt
user1:13812345678:2026-03-16:login_success
user2:15998765432:2026-03-16:login_fail
user3:18611112222:2026-03-17:login_success
```

(3) 匹配包含 user_ 开头的行

```
(kali㉿ 20232311zfz)-[~/Desktop]
$ grep -P 'user_\w+' test_regex.txt
user_admin:020-88889999:2026-03-16:admin_login
```

(4) 匹配包含空格或制表符的行

```
(kali@20232311zfz)-[~/Desktop]
$ grep -P '\s' test_regex.txt
Hello World! 12345 test@example.com
这是中文测试 123abc 空格 制表符 换行
```

(5) 匹配所有以字母 A/B 开头的 Product 行

```
(kali@20232311zfz)-[~/Desktop]
$ grep 'Product_[AB]' test_regex.txt
Product_A:price_999:stock_50
Product_B:price_1999:stock_5
```

2: 量词控制

(1) 匹配 0 个或 1 个连字符（匹配手机号或座机号）

```
(kali@20232311zfz)-[~/Desktop]
$ grep -E '1[3-9][0-9]{9}|020-?[0-9]{8}' test_regex.txt
user1:13812345678:2026-03-16:login_success
user2:15998765432:2026-03-16:login_fail
user3:18611112222:2026-03-17:login_success
user_admin:020-88889999:2026-03-16:admin_login
```

(2) 匹配 6-11 位数字（任意长度数字串）

```
(kali@20232311zfz)-[~/Desktop]
$ grep -E '[0-9]{6,11}' test_regex.txt
user1:13812345678:2026-03-16:login_success
user2:15998765432:2026-03-16:login_fail
user3:18611112222:2026-03-17:login_success
user_admin:020-88889999:2026-03-16:admin_login
```

(3) 贪婪模式：匹配从 price_ 到最后一个数字的所有内容

非贪婪模式：只匹配 price_ 后紧跟的数字 (.*?)

```
(kali@20232311zfz)-[~/Desktop]
$ grep -Po 'price_.*[0-9]' test_regex.txt
price_999:stock_50
price_1999:stock_5
price_299:stock_0

(kali@20232311zfz)-[~/Desktop]
$ grep -Po 'price_.*[0-9]' test_regex.txt
price_9
price_1
price_2
```

3: 位置锚点

行首匹配 (^); 行尾匹配 (\$); 精确匹配 (^+\$); 单词边界 (\b)

```
(kali㉿ 20232311zfz)-[~/Desktop]
$ grep '^user' test_regex.txt
user1:13812345678:2026-03-16:login_success
user2:15998765432:2026-03-16:login_fail
user3:18611112222:2026-03-17:login_success
user_admin:020-88889999:2026-03-16:admin_login

(kali㉿ 20232311zfz)-[~/Desktop]
$ grep 'login_success$' test_regex.txt
user1:13812345678:2026-03-16:login_success
user3:18611112222:2026-03-17:login_success

(kali㉿ 20232311zfz)-[~/Desktop]
$ grep '^Hello.*!$' test_regex.txt

(kali㉿ 20232311zfz)-[~/Desktop]
$ grep '^Hello.*com$' test_regex.txt
Hello World! 12345 test@example.com

(kali㉿ 20232311zfz)-[~/Desktop]
$ grep -P '\bsuccess\b' test_regex.txt

(kali㉿ 20232311zfz)-[~/Desktop]
$ grep -P '\blogin_success\b' test_regex.txt
user1:13812345678:2026-03-16:login_success
user3:18611112222:2026-03-17:login_success
```

4: 分组捕获

分组与逻辑或 (|): 匹配 Product_A 或 Product_B 且价格>100/1000 的行;
提取日期字段 (格式: YYYY-MM-DD)

```
(kali㉿ 20232311zfz)-[~/Desktop]
$ grep -E 'Product_[AB]:price_[1-9][0-9]{2,}' test_regex.txt
Product_A:price_999:stock_50
Product_B:price_1999:stock_5

(kali㉿ 20232311zfz)-[~/Desktop]
$ grep -E 'Product_[AB]:price_[1-9][0-9]{3,}' test_regex.txt
Product_B:price_1999:stock_5

(kali㉿ 20232311zfz)-[~/Desktop]
$ grep -Po '(\d{4}-\d{2}-\d{2})' test_regex.txt
2026-03-16
2026-03-16
2026-03-17
2026-03-16
2026-03-16
```

5: 特殊场景

特殊字符转义:

匹配包含@符号的行 (@无需转义, .需转义);

匹配包含-的行 (-在[]外无需转义)

```
(kali@20232311zfz)-[~/Desktop]  
$ grep 'test@example\.com' test_regex.txt  
Hello World! 12345 test@example.com
```

```
(kali@20232311zfz)-[~/Desktop]  
$ grep '020-' test_regex.txt  
user_admin:020-88889999:2026-03-16:admin_login
```